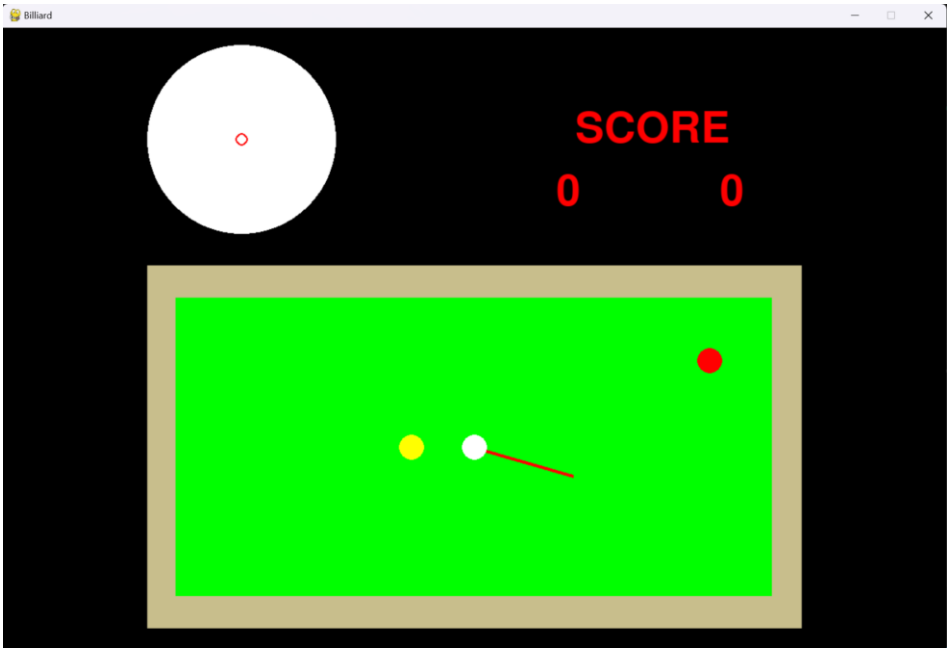
GAME DEVELOPMENT PORTFOLIO

Pygame Billiards

물리 기반 2D 당구 게임 개발

Python 으로 공의 충돌과 회전을 구현하고, 플레이 중 발견한 문제를 추적해 개선한 팀 프로젝트입니다. 팀 프로젝트 종료 후에는 개인 작업으로 메뉴와 화면 크기 설정을 추가했습니다.



플레이 화면 — 스핀 입력 UI 와 점수 표시, 마우스 드래그 샷

프로젝트 한눈에 보기

구분	내용
개발 기술	Python · Pygame · NumPy
주요 기능	샷 입력, 스핀, 공과 벽의 충돌, 3 쿠션 득점 판정
작업 구분	팀 프로젝트 진행 후 개인 기능 개선
소스 코드	github.com/wooing26/pygame_billiards

핵심 경험 — 벡터 연산을 이용한 물리 구현 · 충돌 반복 처리 원인 분석 · 화면과 입력 상태 관리

구현 구조와 게임 진행

Pygame 은 화면 출력과 입력 처리에, NumPy 는 벡터와 물리 연산에 사용했습니다. 단일 파일로 작성된 코드는 역할별 모듈로 분리해 실행 흐름과 수정 위치를 파악하기 쉽도록 정리했습니다.

모듈	담당 기능
main.py	초기화와 메뉴·플레이·설정 화면의 실행 흐름
config.py	화면, 당구대, 공, UI 의 상수와 초기 설정
Ball_class.py	공의 상태와 이동, 마찰, 회전, 충돌 응답
spin.py	타격 위치를 게임 공간의 스핀으로 변환
playing.py	입력 수집, 물리 갱신, 점수와 턴 관리
menu.py / setting.py	메뉴 선택과 화면 크기 설정

한 번의 샷이 처리되는 흐름

1 입력

공을 선택한 뒤 드래그한 거리와 방향으로 초기 속도를 정하고, 별도 UI 에서 스핀을 입력합니다.

2 물리 갱신

이동과 마찰을 적용하고 공 사이의 충돌 및 벽 충돌에 따라 속도와 각속도를 갱신합니다.

3 득점 판정

두 목적구와의 충돌 여부 및 쿠션 횟수를 기록하고, 공이 모두 멈추면 점수를 판단합니다.

4 다음 샷 준비

득점 여부에 따라 턴을 유지하거나 전환한 뒤 충돌 기록과 입력 상태를 초기화합니다.

모듈을 나눈 이유

설정값, 물리 처리, 화면 흐름을 나눠 각 기능의 수정 위치를 구분했습니다. 플레이 진입 시 점수와 충돌 기록을 새로 만들고, 화면 비율은 함수의 인자와 반환값으로 전달했습니다.

게임 환경과 초기 상태 설계

config.py 에는 화면과 당구대의 크기, 색상, 공의 반지름과 질량, 초기 위치, UI 기준점 등 여러 모듈에서 사용하는 값을 모았습니다. 단일 파일에 흩어져 있던 설정을 분리하면서 화면 구성과 물리 처리에 필요한 기준을 정리했습니다.

실제 치수를 기준으로 한 배치

당구대의 가로·세로는 실제 치수를 기준으로 1/3 비율을 적용했습니다. 공의 지름에도 같은 비율을 사용하고, 질량에는 부피 비율인 1/27 을 적용했습니다. 세 공의 시작 위치를 설정값으로 두어 게임을 다시 시작할 때 같은 배치로 초기화할 수 있도록 했습니다.

```
29  # 당구대 설정
30  height = 1422 / 3
31  width = 2844 / 3
32  height_out = 1730 / 3
33  width_out = 3120 / 3
34  midpoint = [back_size[0] / 2, back_size[1] * 2 / 3]
35
36
37  # 당구대 안에서만 공이 움직이게 크기 설정
38  x_lim = np.array([midpoint[0] - width / 2, midpoint[0] + width / 2])
39  y_lim = np.array([midpoint[1] - height / 2, midpoint[1] + height / 2])
40  x_out = np.array([midpoint[0] - width_out / 2, midpoint[0] + width_out / 2])
41  y_out = np.array([midpoint[1] - height_out / 2, midpoint[1] + height_out / 2])
42
43
44  # 당구대
45  pool_wall = np.array([x_lim, y_lim])
```

config.py 의 당구대 크기와 경계 좌표 설정

당구대 중심을 기준으로 한 UI

당구대 중심인 midpoint 를 기준으로 화면 요소의 위치를 계산했습니다. 당구대 위치가 달라질 때 점수 표시와 스핀 입력 UI 도 같은 기준에 따라 이동하도록 구성했습니다. 점수 표시는 함수에서 현재 값을 받아 새로 그리는 방식입니다.

설정	적용 목적
기본 화면 크기와 FPS	창 크기와 게임 루프의 목표 실행 속도 설정
반지름과 질량	충돌 판정과 충격량 및 회전 계산에 사용
speed_limit	드래그 입력으로 지나치게 강한 샷이 발생하는 것을 제한
frame_skip	공의 쌍별 충돌 응답이 반복되는 현상을 완화

화면 비율 변경 시 질량까지 함께 바꾸는 구현은 표시 크기와 물리 상태를 연결합니다. 물리적 유사성 전체를 보장하는 모델은 아니므로, 화면 크기별 운동 일관성은 추가 검증이 필요한 부분입니다.

공의 상태와 구름 운동

Ball 클래스에서 위치와 속도는 2 차원 벡터, 스핀은 3 차원 벡터로 관리했습니다. 평면 위의 이동과 회전 운동을 연결하면서 공마다 같은 물리 처리 코드를 재사용하도록 구성했습니다.

이동과 마찰

속도의 반대 방향으로 감속을 적용한 뒤 위치를 갱신했습니다. 작은 속도나 각속도가 계속 남는 현상은 임계값 이하에서 0 으로 처리했습니다. 회전 감쇠와 스핀 영향 계수는 플레이하며 조정했습니다.

회전 상태를 포함한 공 객체

공은 위치·속도·스핀·색상·반지름·질량·마찰 값·관성모멘트를 가집니다. 위치·속도·스핀은 @property 로 접근하도록 구성했습니다. 구의 관성모멘트 $I = (2/5)mr^2$를 사용해 회전에 필요한 기준값을 설정했습니다. 클래스 구조는 공 객체를 추가하기 쉽지만, 4 구 등으로 확장할 때는 득점 규칙도 별도로 바꿔야 합니다.

```
133     # 바닥과의 상호작용 (백스핀)
134     def rolling(self, c = 0.0003, mu = 0.0005):
135         # 각속도 감소
136         if np.any(self.spin != 0):
137             self.spin -= self.spin / np.linalg.norm(self.spin) * 5  #프레임마다 일정 각속도 계속 감소
138             if np.linalg.norm(self.spin) < 5:
139                 self.spin = np.array([0.0, 0.0, 0.0])
140 
141         #각속도 추가 항
142         n_hat = np.array([0, 0, 1])
143         self.spin -= c * np.cross(self.v, n_hat)
144 
145         #각속도에 의한 중심속도 변화
146         dv = -mu * np.cross(self.spin, n_hat)
147         self.v[0] += dv[0]
148         self.v[1] += dv[1]
```

rolling 함수의 회전 감쇠와 구름 운동 처리

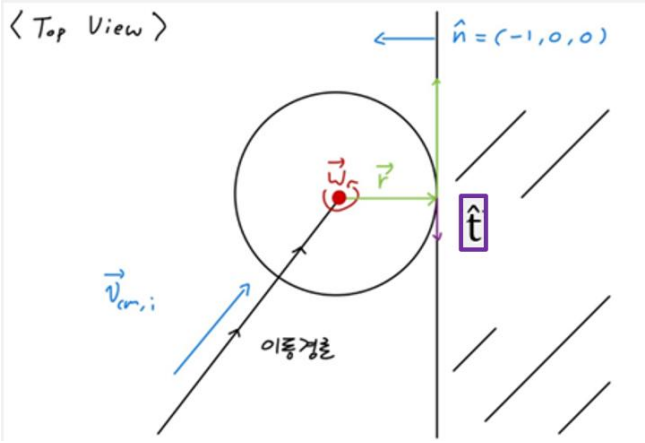
구름과 백스핀의 연결

바닥 법선과 선속도의 외적으로 구름에 따른 회전 방향을 정하고, 조정 계수를 곱해 각속도에 반영했습니다. 반대로 각속도와 바닥 법선의 외적으로 중심 속도의 변화 방향을 계산해 백스핀 효과를 구현했습니다. 즉 이동이 회전을 만들고, 회전도 다시 이동에 영향을 주도록 구성했습니다.

회전 감쇠에는 플레이하면서 정한 감소량을 사용했습니다. 이 값과 구름·백스핀 계수는 정밀한 물성 측정값이 아니라 게임의 움직임을 조정하기 위한 값입니다.

벽 충돌과 스핀 반응 계산

벽에 부딪힐 때 단순히 속도의 부호만 뒤집으면 회전에 따른 접선 방향의 변화가 드러나지 않습니다. change_velocity 함수에서는 공 중심의 이동과 접촉점의 회전을 함께 고려해 충돌 이후의 속도와 각속도를 계산했습니다.



벽의 법선과 접촉점에서의 공 운동

접촉점 속도와 충격량

중심 속도 v_cm 에 회전 성분 ω × r 을 더해 접촉점 속도를 계산했습니다. 여기서 r 은 공 중심에서 접촉점까지의 벡터입니다. 평면상의 속도에 회전 연산을 적용할 때는 z 성분을 추가하거나 제거해 2 차원과 3 차원 표현을 맞췄습니다.

v_contact = v_cm + ω × r

벽의 법선 방향에는 반발계수를 이용한 충격량을, 접선 방향에는 마찰에 따른 충격량을 적용했습니다. 접선 충격량은 마찰 한계 안으로 제한하고, 두 성분을 합쳐 중심 속도와 회전 상태를 갱신하는 흐름으로 작성했습니다.

위치 보정으로 벽 관통 방지

벽과의 접촉은 공 중심에 반지름을 더한 경계 검사로 판단했습니다. 프레임 단위 이동으로 중심이 경계를 넘어가면, 충돌 응답과 함께 중심을 벽에서 반지름만큼 떨어진 위치로 되돌렸습니다. 속도 변경과 위치 보정이 각각 다른 역할을 하도록 처리했습니다.

원문 수식의 접선 충격량 유도와 회전항 단순화는 추가 검증이 필요한 부분입니다. 여기서는 구현의 계산 흐름을 설명하며, 해당 유도를 검증된 물리 모델로 단정하지 않습니다.

공 사이 충돌 판정과 속도 갱신

공 사이의 충돌은 overlaps 로 접촉 여부를 검사하고, collision 에서 속도를 갱신하도록 구분했습니다. 같은 판정은 물리 응답뿐 아니라 득점에 필요한 목적구 접촉 기록에도 사용했습니다.

원의 겹침으로 충돌 판정

두 공의 중심 사이 거리와 반지름의 합을 비교했습니다. 거리가 더 작으면 겹침 상태로 보고 충돌 응답을 수행합니다. 두 중심을 잇는 벡터를 정규화해 충돌 법선 n 을 구하고, 상대속도의 법선 성분을 충격량 계산에 사용했습니다.

```
157     # 공끼리 충돌시 움직임
158     def collision(self, other, e_b = 0.95): #공사이 반발계수
159         # 충돌했을 때의 속도 변화 함수 작성
160         def change_velocity():
161             m1, m2 = self.m, other.m
162             M = m1 + m2
163             p1, p2 = self.pos, other.pos
164             dis = np.linalg.norm(p1 - p2)
165             pos_diff = p2 - p1
166             n_hat = pos_diff / dis #충돌면 단위 벡터
167             v1, v2 = self.v, other.v
168             j = -(1+e_b) * np.dot((v1 - v2), n_hat) / (1 / m1 + 1 / m2) #충격량
169             u1 = v1 + j / m1 * n_hat
170             u2 = v2 - j / m2 * n_hat
171             self.v = u1
172             other.v = u2
173 
174         # 충돌시 움직임 변화
175         if self.overlaps(other):
176             change_velocity()
```

collision 함수의 법선 계산과 두 공의 속도 갱신

같은 충격량을 반대 방향으로 적용

공 중심과 접촉점을 잇는 방향이 법선과 나란한 구형 모델을 사용했습니다. 상대속도와 반발계수 e, 두 질량 m1·m2 로 충격량 j 를 계산하고, 각 공의 속도에 반대 부호로 반영했습니다.

$$j = -(1 + e)((v1 - v2) \cdot n) / (1/m1 + 1/m2)$$
$$u1 = v1 + (j/m1)n \quad u2 = v2 - (j/m2)n$$

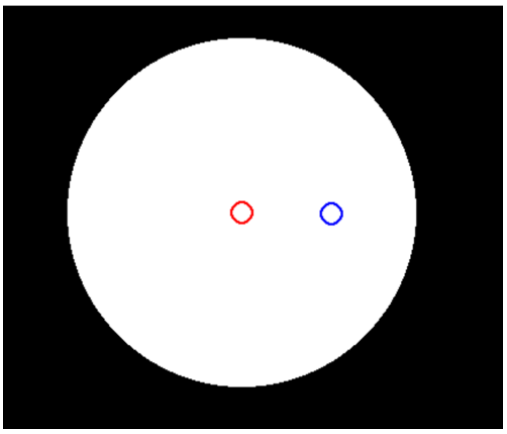
위 식은 n 의 방향을 두 공에 대해 일관되게 정한다는 전제의 표기입니다. 실제 코드 설명에서는 공 사이 충돌과 벽 충돌 함수를 구분했습니다. 공 사이의 회전 마찰 전달까지 모두 구현한 모델로 확대 해석하지 않았습니다.

쌍별 처리와 반복 응답 문제

세 공의 서로 다른 조합은 세 쌍입니다. 공이 n 개라면 조합은 n(n−1)/2 개이며, 방향을 구분한 호출 수와는 다릅니다. 겹침 상태가 이어지면 같은 쌍에 충돌 응답이 반복되는 문제가 있어, 뒤의 오류 해결 항목처럼 쌍별 카운터를 추가했습니다.

스핀 입력과 좌표 변환

단순한 방향과 세기 입력에 더해, 공의 어느 지점을 타격하는지 선택할 수 있도록 스핀 UI 를 구성했습니다. 선택한 타격 위치를 샷 방향에 맞는 게임 공간의 각속도로 변환하는 것이 핵심입니다.



스핀 입력 UI 빨간 점은 중심, 파란 점은 마우스 위치

문제와 변경 과정

타격 UI 는 샷 방향에서 공을 바라본 단면이므로, 당구대 화면과 좌표축이 다릅니다. 초기에는 두 방향 벡터의 내적과 arccos 로 각도를 구했지만, 같은 벽에 충돌해도 회전 반응이 일관되지 않았습니다.

이후에는 정규화한 샷 방향의 x·y 성분을 직접 이용하도록 변경했습니다. UI 의 세로축을 게임 공간의 z 축에 대응시키고, Pygame 의 화면 y 축이 아래쪽으로 증가하는 방향도 반영했습니다.

```python
# 스핀 입력
def Get_spin(mouse_pos, v_cm, spin_ball):
    r_impact = [0, 0, 0]
    v_cm = v_cm / np.linalg.norm(v_cm)
    r_hat = [mouse_pos[0] - spin_ball[0], mouse_pos[1] - spin_ball[1]]
    r_impact[2] = r_hat[1]
    r_impact[0], r_impact[1] = -r_hat[0] * v_cm[1], r_hat[0] * v_cm[0]
    return np.array(r_impact) #r_impact는 큰 값
```

spin.py 의 입력 좌표 변환 코드

입력 크기 보정

스핀 입력용 공과 실제 플레이 공은 크기가 다르므로 반지름 비율로 입력값을 보정했습니다. 그 후 추가 계수를 적용해 작은 마우스 이동으로 과도한 회전이 발생하지 않도록 조정했습니다.

좌표 변환에서 배운 점

회전각만 구하는 것보다 기준축과 부호, 입력 좌표가 의미하는 방향을 먼저 정의하는 것이 중요했습니다. 팀원들과 좌표계와 계산 방식을 함께 검토하며 문제를 해결했습니다.

샷 입력과 3 쿠션 득점 상태 관리

playing.py 는 마우스 입력과 물리 갱신, 샷 종료 후의 득점 판정을 연결합니다. 모든 공이 정지한 상태에서 새 입력을 받고, 움직이는 동안에는 충돌 기록을 쌓은 뒤 다시 정지하면 샷을 정리하도록 구성했습니다.

드래그로 방향과 세기 결정

Pygame 이벤트에서 마우스를 누른 좌표와 놓은 좌표를 받아 차이를 초기 속도로 사용했습니다. 공을 선택한 경우에만 반응하고, 드래그 길이에 조정 비율을 적용한 뒤 speed_limit 으로 최대 속력을 제한했습니다. 초기 속도와 스핀을 선택하면 입력 표시를 유지해 확인할 수 있도록 했습니다.

충돌 순서를 반영하는 기록

상태	의미와 갱신 방식
count[0]	첫 번째 목적구와 충돌하면 1 로 기록
count[1]	두 번째 목적구와 충돌하면 1 로 기록
count[2]	쿠션 횟수를 최대 3 까지 기록하며 두 목적구를 모두 맞힌 뒤에는 증가시키지 않음

예를 들어 두 목적구를 먼저 맞힌 뒤 쿠션에 세 번 닿는 상황은 득점으로 처리하지 않도록 했습니다. 원문 구현에서는 모든 공이 멈췄을 때 count == [1, 1, 3]인지 검사해 해당 플레이어의 점수를 1 점 올렸습니다.

```
179     # 충돌 횟수 세기 (3구의 3쿠션)
180     def count_score(self, other1, other2, count, pool_wall):
181
182         if self.overlaps(other1) and count[0] == 0:
183             count[0] += 1
184         if self.overlaps(other2) and count[1] == 0:
185             count[1] += 1
186         if self.limit_x0(pool_wall) and count[2] < 3 and count[:2] != [1, 1]:
187             count[2] += 1
188         if self.limit_x1(pool_wall) and count[2] < 3 and count[:2] != [1, 1]:
189             count[2] += 1
190         if self.limit_y0(pool_wall) and count[2] < 3 and count[:2] != [1, 1]:
191             count[2] += 1
192         if self.limit_y1(pool_wall) and count[2] < 3 and count[:2] != [1, 1]:
193             count[2] += 1
```

count_score 함수의 목적구 충돌 및 쿠션 횟수 기록

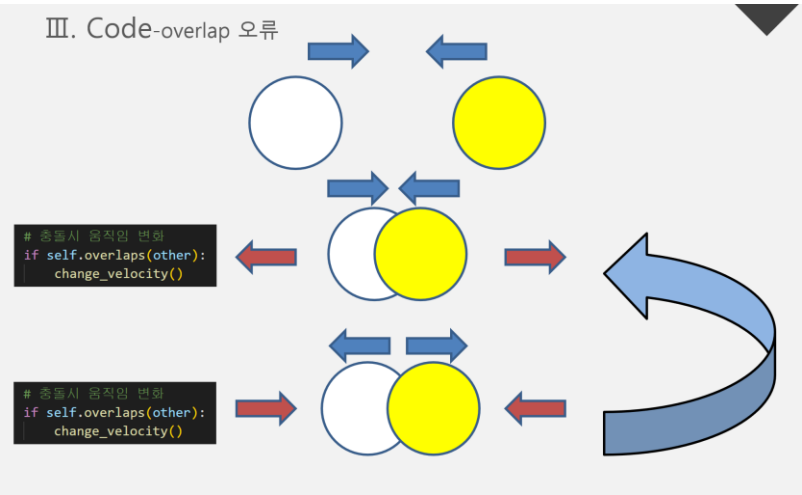
한 번의 샷을 한 번만 판정

정지 상태가 여러 프레임 지속되어도 점수와 턴이 반복해서 바뀌지 않도록 count_num 으로 판정 단계와 입력 단계를 분리했습니다. 득점하면 현재 턴을 유지하고, 턴을 넘길 때는 (player_turn + 1) % 2 로 전환했습니다. 판정 후에는 충돌 기록과 관련 상태를 초기화했습니다.

fps.tick(60)은 게임 루프를 목표 60FPS 로 제한하는 용도입니다. 실행 환경과 관계없이 매초 정확히 60 회 갱신된다는 보장은 없으므로, 프레임 시간에 따른 물리 갱신은 향후 개선 방향으로 구분했습니다.

공이 붙어서 이동하는 오류 해결

충돌한 두 공이 떨어지지 않고 붙은 채 이동하는 현상이 발생했습니다. FPS 를 높이거나 속력을 낮추는 초기 시도만으로는 문제가 사라지지 않아, 상태 출력 로그와 반복 실행으로 원인을 추적했습니다.



충돌 후 겹침이 남아 충돌 응답이 반복되는 상황

원인

프레임 단위로 위치를 갱신하면 충돌 직후에도 공이 겹쳐 있을 수 있습니다. 겹침 여부만으로 다음 프레임에 충돌 응답을 다시 적용하면, 이미 멀어지는 공의 속도가 다시 바뀌면서 붙어 움직이는 것처럼 보였습니다.

적용한 해결

공의 쌍마다 frame_count 를 두고, 충돌 직후에는 frame_skip 값인 3 프레임 동안 재처리를 건너뛰었습니다. 카운터를 매 프레임 줄인 뒤 0 이 되면 다시 충돌을 처리하도록 구성했습니다.

공의 쌍마다 관리한 이유

흰 공과 노란 공, 노란 공과 빨간 공, 빨간 공과 흰 공의 접촉은 서로 다른 사건입니다. frame_count 의 각 요소가 한 쌍을 담당하도록 하여, 어느 두 공이 충돌했는지 구분하고 각각의 대기 시간을 줄였습니다.
FPS 나 속력만 바꾸는 시도에서 충돌 처리 시점 자체를 제어하는 방식으로 바꾼 것입니다.

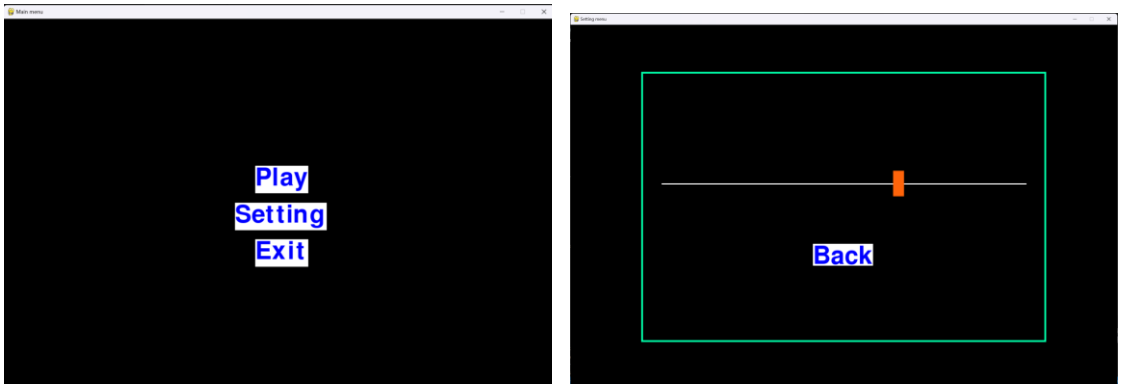
한계와 다음 개선

프레임을 건너뛰는 방식은 반복 응답을 완화하는 처리입니다. 속도와 접촉 조건에 따라 한계가 있으므로, 향후에는 상대속도로 접근 여부를 판정하고 공의 겹친 위치를 보정하는 방식으로 개선할 수 있습니다.

문제 해결 경험 재현 상황 관찰 → 상태 로그 추적 → 반복 응답 원인 확인 → 공의 쌍별 재처리 간격 적용

메뉴와 화면 크기 설정 개선

팀 프로젝트 종료 후, 다양한 모니터에서 게임을 사용할 수 있도록 메뉴와 화면 크기 설정을 개인 작업으로 추가했습니다. 이 과정에서 파일 구조도 역할별로 정리했습니다.



메인 메뉴와 화면 크기 설정 화면

비율을 미리 보고 적용하는 설정

주황색 슬라이더를 움직이면 초록색 미리보기 사각형이 바뀌고, Back 버튼으로 선택한 크기를 적용하도록 했습니다. screen_ratio 는 슬라이더의 오른쪽 끝에서 1, 왼쪽 끝에서 2 가 되며, 창과 화면 요소를 이 값으로 나눠 축소했습니다.

설정값 전달과 입력 개선

화면 비율은 각 함수의 인자와 반환값으로 전달해 메뉴와 플레이에 적용했습니다. 슬라이더를 빠르게 움직일 때 입력이 끊기는 문제는 마우스 좌표 반영과 영역 검사 순서를 조정해 해결했습니다.

화면·당구대·공·UI 에 같은 비율을 반영했으며, 질량에는 비율의 세제곱을 적용했습니다. 표시 크기 변경 전후의 물리 일관성은 추가 검증이 필요한 부분입니다.

메뉴 전환과 재시작

menu·play·set 상태에 따라 해당 화면을 실행하고, 창 닫기 이벤트에서는 상태를 종료하도록 구성했습니다. main.py 는 pygame.init()과 세 공의 초기화를 담당하고, 설정값을 각 화면에 전달했습니다. 플레이가 끝나면 위치와 속도를 초기화해 다음 실행에 이전 샷의 상태가 남지 않도록 했습니다.

협업 과정과 개발을 통해 배운 점

이 프로젝트에서는 물리 계산을 실제 게임 동작으로 연결하는 작업과, 팀원들의 경험 차이에 맞춰 함께 구현하는 작업을 경험했습니다. 충돌과 스핀 문제를 해결할 때는 혼자 가설을 좁히는 방식에서 관찰 결과를 공유하고 함께 검토하는 방식으로 바꿨습니다.

경험 차이를 고려한 역할 분담

팀원들의 프로그래밍 경험을 고려해 필요한 코드 흐름을 설명하고 구현, 물리 이론 조사, 발표 준비를 나눴습니다. 각자가 맡은 부분을 직접 발표하도록 준비해 담당 영역에 대한 이해를 높였습니다.

팀 작업과 개인 작업의 구분

팀 프로젝트에서는 공의 운동과 스핀, 오류 원인을 함께 논의했습니다. 팀원에게 필요한 부분의 코드 흐름을 설명하고, 구현 가능한 기능과 이론 조사 역할을 나눴습니다. 프로젝트 종료 후 메뉴·설정 추가와 화면 비율 전달 구조의 개선은 개인 작업으로 진행했습니다.

새로운 도구를 적용한 경험

당시 처음 다룬 NumPy 로 벡터의 정규화, 내적·외적과 좌표 변환을 구현했습니다. 수식의 의미를 코드의 배열 차원과 부호에 맞추는 과정에서, 계산식만큼 좌표계와 데이터 표현을 명확히 해야 한다는 점을 배웠습니다.

개발을 통해 배운 점

- 문제가 발생하는 조건과 상태 변화를 기록해야 원인에 맞는 수정으로 이어질 수 있었습니다.
- 좌표계와 상태 전환을 명확히 정의하는 일이 물리 계산만큼 중요했습니다.
- 혼자 해결하기 어려운 문제는 관찰 결과와 가설을 공유하면서 팀의 아이디어를 모았습니다.

향후 개선 방향

개선 항목	검토할 내용
충돌 안정성	상대속도로 접근 여부를 판정하고 겹친 위치를 보정
물리 단위와 화면 크기	렌더링 비율과 물리 상태를 분리해 화면별 운동 일관성 확인
시간과 상태 관리	프레임 시간 기반 갱신과 샷 종료 상태의 명확한 구분
검증과 유지보수	충돌 방향, 스핀 입력, 재시작 등 주요 상황의 재현 절차 정리